PRACTICAL 6

JSON Files, Fetch API, Dynamic Rendering & Data Operations

Student Name	KIRTAN KAKADIYA
Enrollment / Roll No.	25CS026
Program / Semester	3RD
Course / Subject	WEB DEVELOPMENT FRAMEWORK
Faculty Name	BRINDA MAM
Submission Date	14-10-2026

1. Aim

To create a JavaScript-based webpage that fetches JSON data, renders records dynamically, and supports searching, filtering, sorting, pagination, loading/error states, and modular JavaScript.

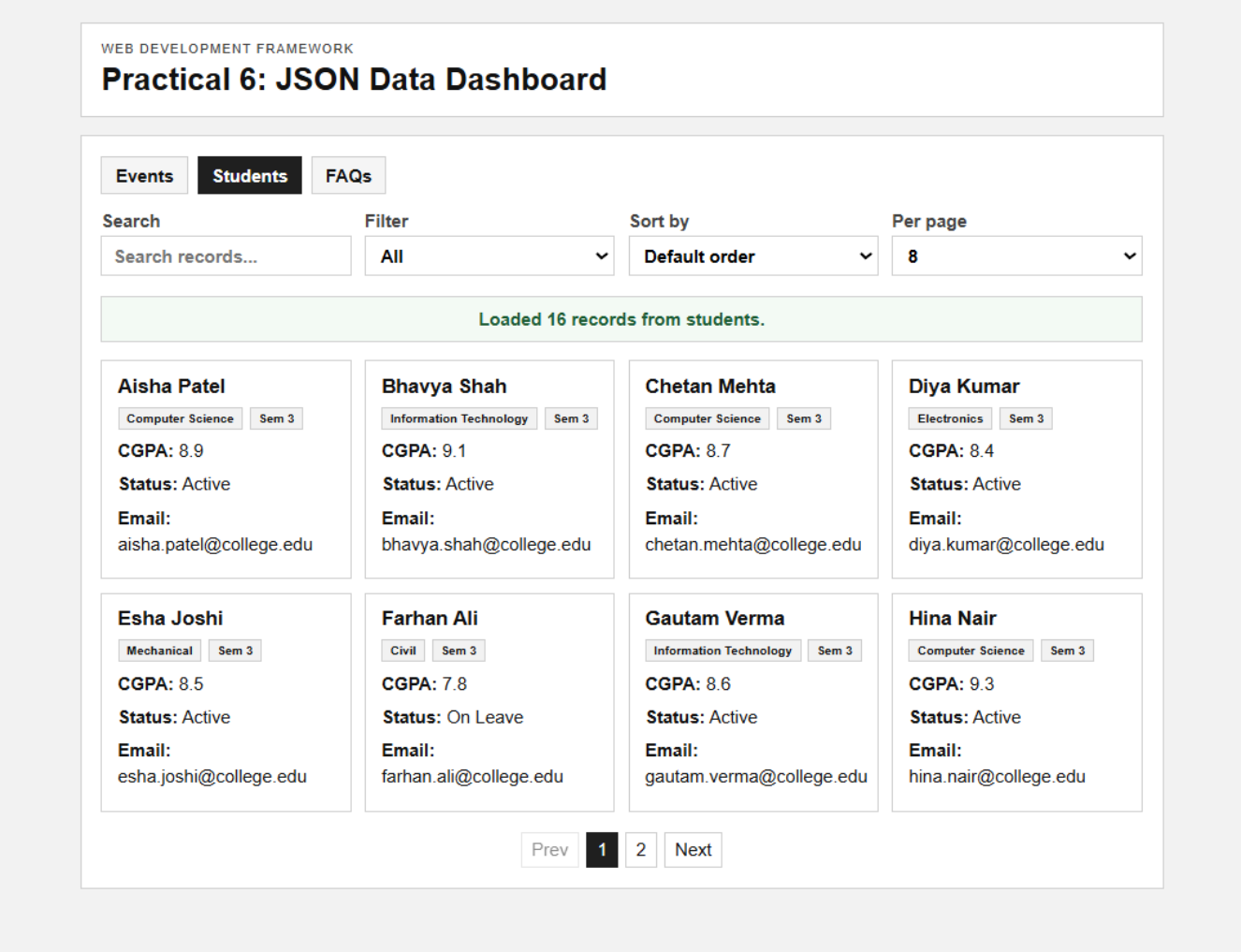
2. Main Submission Requirements

- JSON Files – Create/use events.json, students.json and faqs.json, with at least 15 records in each file.
- Fetch API – Fetch JSON data using JavaScript.
- Dynamic Rendering – Display JSON data dynamically on the webpage.
- Search & Filter – Add search and filter functionality.
- Sorting – Sort records using JavaScript.
- Pagination – Display records page by page.
- Loading/Error – Show a loading message and handle errors.
- Modular JavaScript – Keep JavaScript code properly organized.

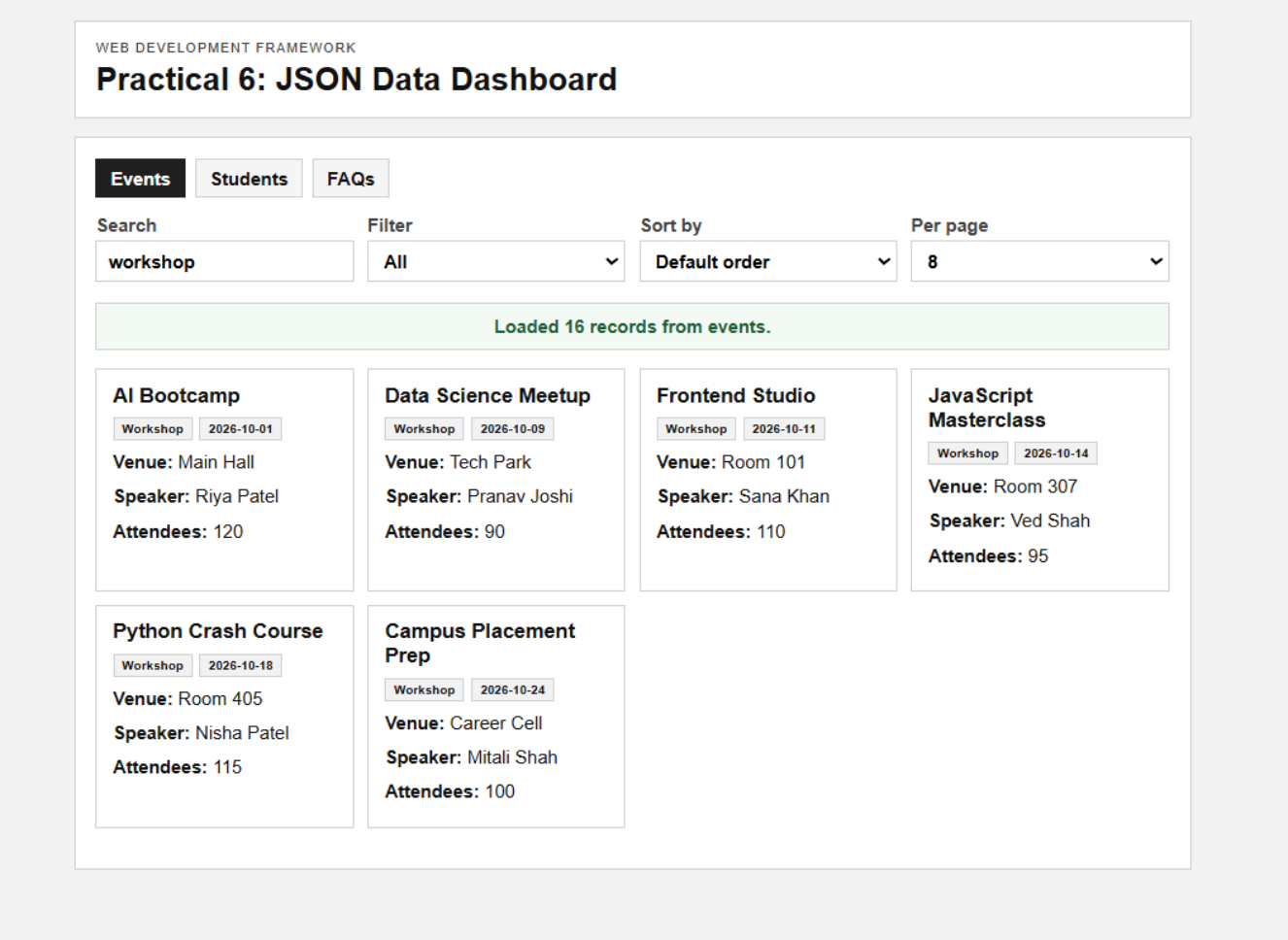
3. Required Screenshots

Paste one clear screenshot into each placeholder. Ensure the relevant data or UI state is visible and readable.

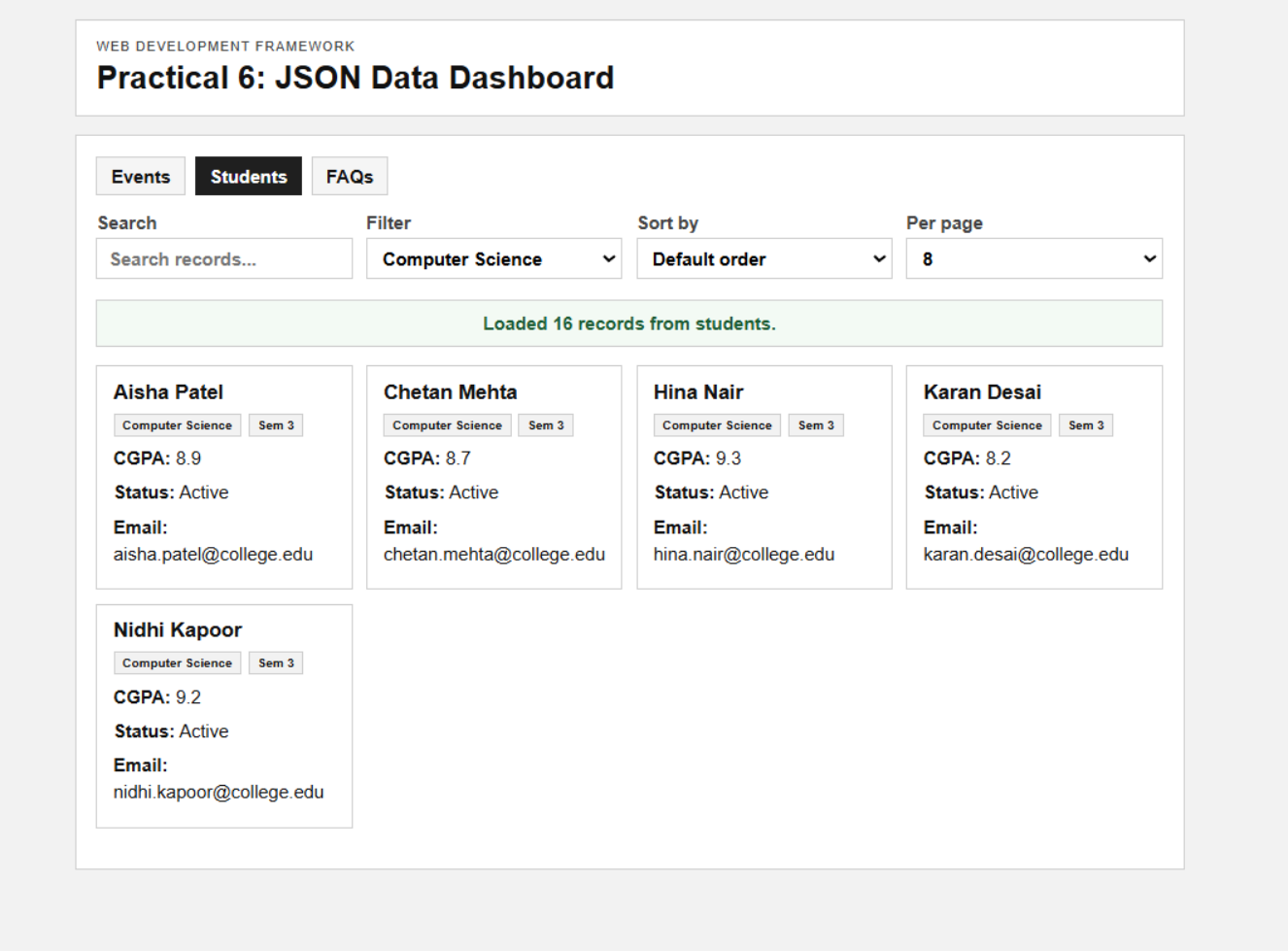
3.1 JSON Data Displayed



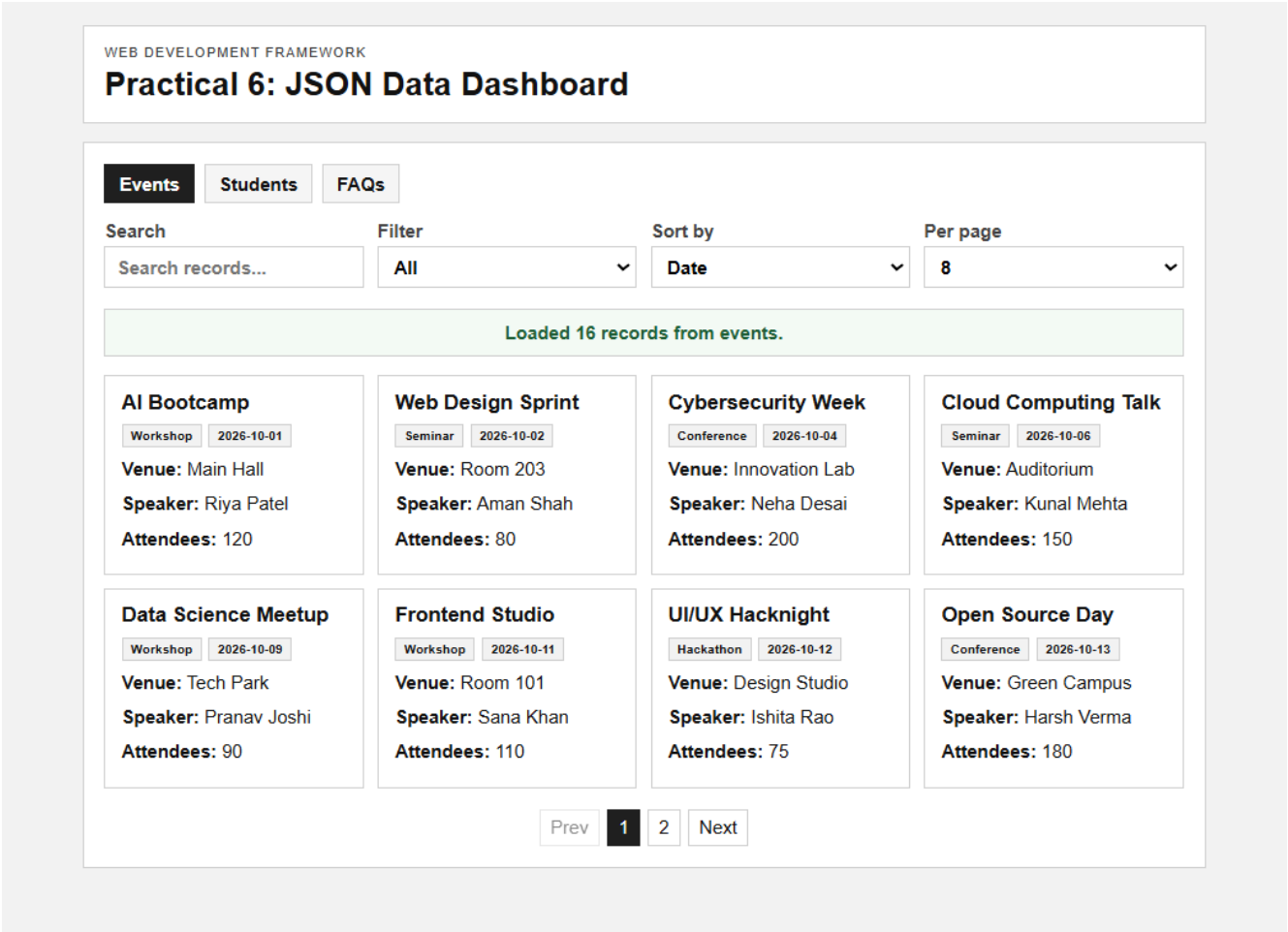
3.2 Search Result



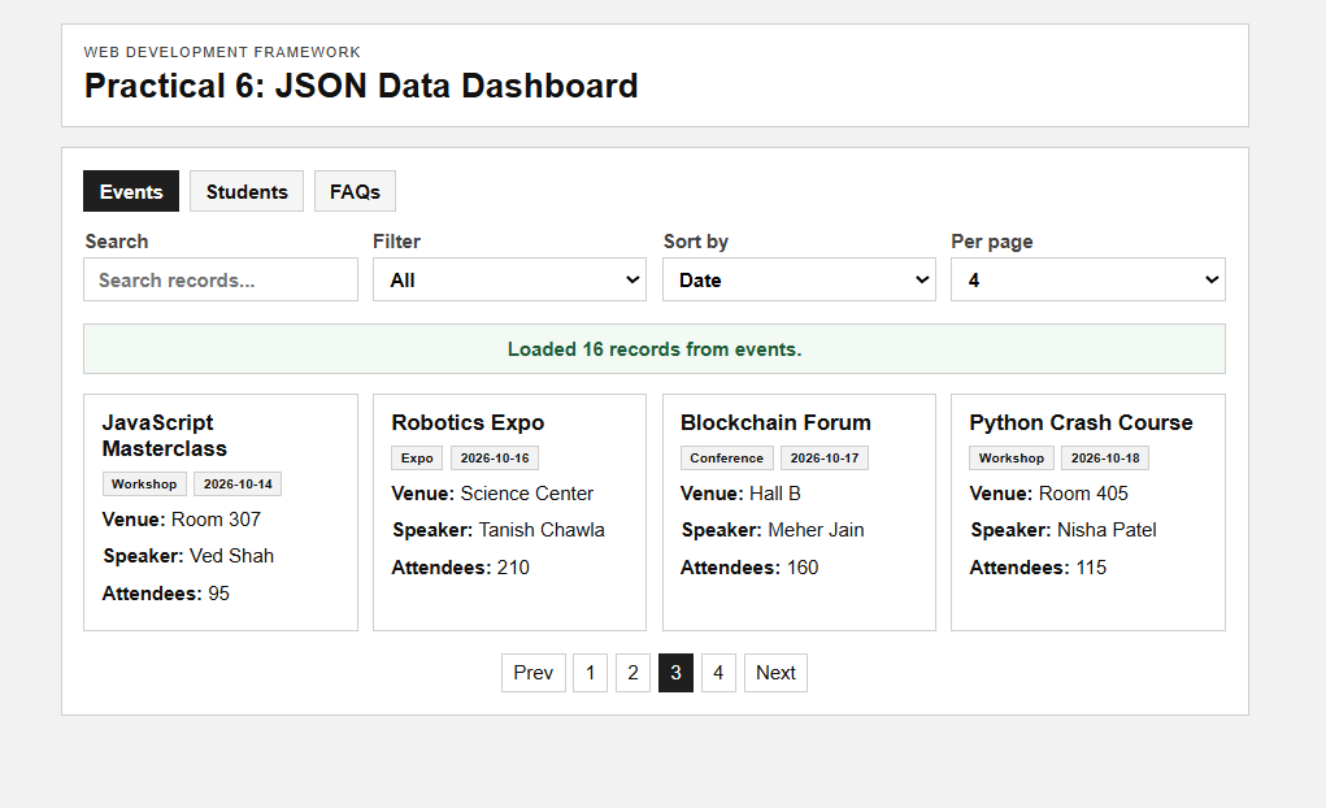
3.3 Filter Result



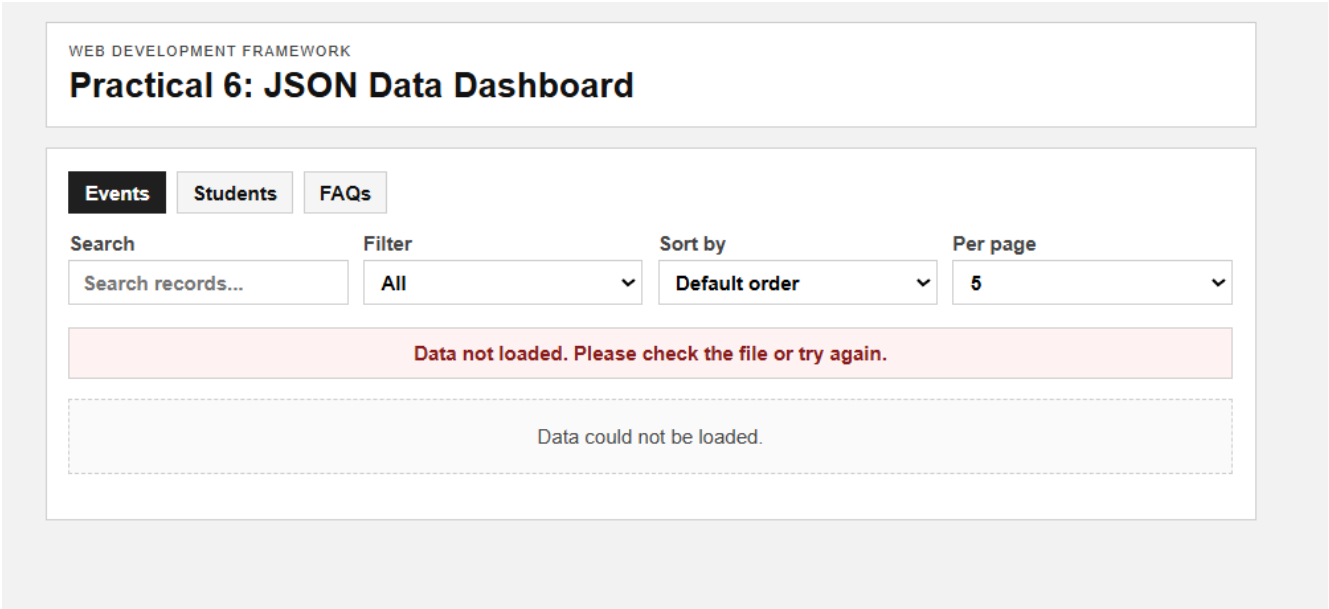
3.4 Sorting Result



3.5 Pagination



3.6 Loading / Error Message




```
1  const DATASET_FILES = {
2    events: './data/eventss.json',
3    students: './data/students.json',
4    faqs: './data/faqs.json'
5  };
6
7  export async function fetchDataset(name) {
8    const path = DATASET_FILES[name];
9    if (!path) {
10     throw new Error(`Unknown dataset: ${name}`);
11   }
12
13   const response = await fetch(path);
14   if (!response.ok) {
15     throw new Error(`Unable to load ${name}.`);
16   }
17
18   return response.json();
19 }
20
21 export function getFilterValues(items, field) {
22   return [...new Set(items.map((item) => item[field]).filter(Boolean))].sort();
23 }
24
25 export function getSortOptions(datasetName) {
26   const common = [
27     { value: 'default', label: 'Default order' }
28   ];
```

As, intentionally changed path of events json file from /events.json to /events.json

4. Recommended Technical Evidence

These additional screenshots are recommended if the practical is graded against all eight main submission points.

4.1 JSON Files – Record Count

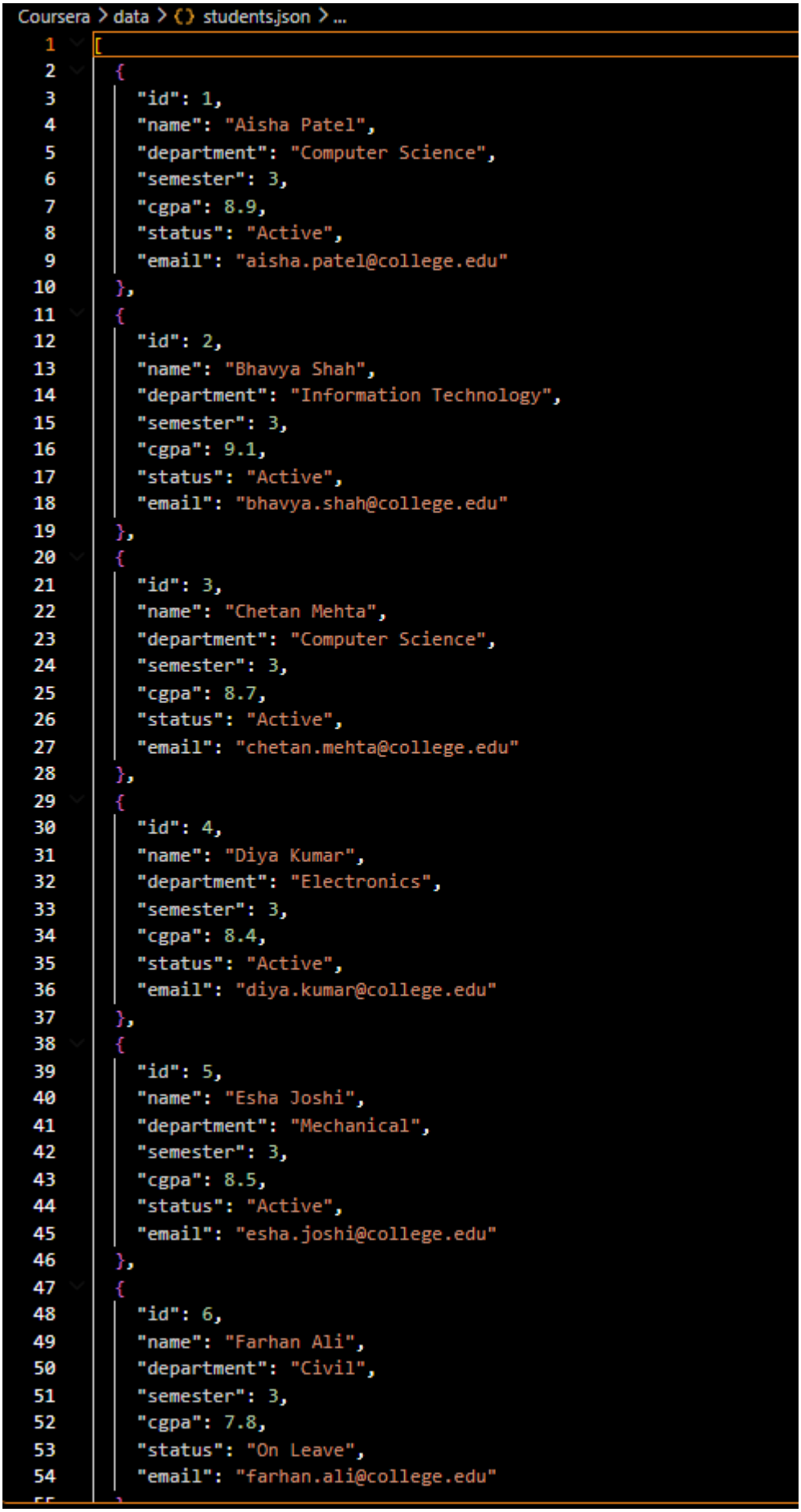
Coursera > data > {} events.json > {} 1


```json
[
  {
    "id": 1,
    "title": "AI Bootcamp",
    "date": "2026-10-01",
    "venue": "Main Hall",
    "category": "Workshop",
    "speaker": "Riya Patel",
    "attendees": 120
  },
  {
    "id": 2,
    "title": "Web Design Sprint",
    "date": "2026-10-02",
    "venue": "Room 203",
    "category": "Seminar",
    "speaker": "Aman Shah",
    "attendees": 80
  },
  {
    "id": 3,
    "title": "Cybersecurity Week",
    "date": "2026-10-04",
    "venue": "Innovation Lab",
    "category": "Conference",
    "speaker": "Neha Desai",
    "attendees": 200
  },
  {
    "id": 4,
    "title": "Cloud Computing Talk",
    "date": "2026-10-06",
    "venue": "Auditorium",
    "category": "Seminar",
    "speaker": "Kunal Mehta",
    "attendees": 150
  },
  {
    "id": 5,
    "title": "Data Science Meetup",
    "date": "2026-10-09",
    "venue": "Tech Park",
    "category": "Workshop",
    "speaker": "Pranav Joshi",
    "attendees": 90
  },
  {
    "id": 6,
    "title": "Frontend Studio",
    "date": "2026-10-11",
    "venue": "Room 101",
    "category": "Workshop",
    "speaker": "Sana Khan",
    "attendees": 110
```

Coursera > data > {} faqs.json > ...


```json
[
  {
    "id": 1,
    "category": "Basics",
    "question": "What is JSON?",
    "answer": "JSON is a lightweight text format used to store and exchange data."
  },
  {
    "id": 2,
    "category": "Basics",
    "question": "What does Fetch API do?",
    "answer": "It allows JavaScript to request data from a server without reloading the page."
  },
  {
    "id": 3,
    "category": "Frontend",
    "question": "Can JavaScript render data dynamically?",
    "answer": "Yes, JavaScript can create HTML from JSON objects and insert it into the page."
  },
  {
    "id": 4,
    "category": "Frontend",
    "question": "What is a search filter?",
    "answer": "A search filter narrows records based on user-entered text or a selected field value."
  },
  {
    "id": 5,
    "category": "Backend",
    "question": "Where are JSON files stored?",
    "answer": "They are usually stored in a folder such as data/ and loaded with fetch()."
  },
  {
    "id": 6,
    "category": "Data",
    "question": "Why use pagination?",
    "answer": "Pagination breaks large data sets into manageable pages to improve usability."
  },
  {
    "id": 7,
    "category": "Frontend",
    "question": "How is sorting implemented?",
    "answer": "JavaScript compares values on a field and rearranges the array before rendering."
  },
  {
    "id": 8,
    "category": "Basics",
    "question": "Is JSON strict about quotation marks?",
    "answer": "Yes, keys and string values are enclosed in double quotes in valid JSON."
  },
  {
    "id": 9,
    "category": "Frontend",
    "question": "What is a loading state?",
    "answer": "A loading state shows users that data is being fetched before the content appears."
```

Coursera > data > {} students.json > ...



```
[
  {
    "id": 1,
    "name": "Aisha Patel",
    "department": "Computer Science",
    "semester": 3,
    "cgpa": 8.9,
    "status": "Active",
    "email": "aisha.patel@college.edu"
  },
  {
    "id": 2,
    "name": "Bhavya Shah",
    "department": "Information Technology",
    "semester": 3,
    "cgpa": 9.1,
    "status": "Active",
    "email": "bhavya.shah@college.edu"
  },
  {
    "id": 3,
    "name": "Chetan Mehta",
    "department": "Computer Science",
    "semester": 3,
    "cgpa": 8.7,
    "status": "Active",
    "email": "chetan.mehta@college.edu"
  },
  {
    "id": 4,
    "name": "Diya Kumar",
    "department": "Electronics",
    "semester": 3,
    "cgpa": 8.4,
    "status": "Active",
    "email": "diya.kumar@college.edu"
  },
  {
    "id": 5,
    "name": "Esha Joshi",
    "department": "Mechanical",
    "semester": 3,
    "cgpa": 8.5,
    "status": "Active",
    "email": "esha.joshi@college.edu"
  },
  {
    "id": 6,
    "name": "Farhan Ali",
    "department": "Civil",
    "semester": 3,
    "cgpa": 7.8,
    "status": "On Leave",
    "email": "farhan.ali@college.edu"
```

4.2 Fetch API Implementation

Coursera > js > dataService.js > DATASET_FILES > events

```
const DATASET_FILES = {
  events: './data/events.json',
  students: './data/students.json',
  faqs: './data/faqs.json'
};

export async function fetchDataset(name) {
  const path = DATASET_FILES[name];
  if (!path) {
    throw new Error(`Unknown dataset: ${name}`);
  }

  const response = await fetch(path);
  if (!response.ok) {
    throw new Error(`Unable to load ${name}.`);
  }

  return response.json();
}

export function getFilterValues(items, field) {
  return [...new Set(items.map((item) => item[field]).filter(Boolean))].sort();
}

export function getSortOptions(datasetName) {
  const common = [
    { value: 'default', label: 'Default order' }
  ];

  const datasetOptions = {
    events: [
      { value: 'title', label: 'Title' },
      { value: 'date', label: 'Date' },
      { value: 'category', label: 'Category' }
    ],
    students: [
```

4.3 Dynamic Rendering

```
function escapeHtml(value) {
  return String(value ?? '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

function renderEvents(item) {
  return `
    <div class="record-card">
      <h3>${escapeHtml(item.title)}</h3>
      <div class="record-meta">
        <span class="chip">${escapeHtml(item.category)}</span>
        <span class="chip">${escapeHtml(item.date)}</span>
      </div>
      <p><strong>Venue:</strong> ${escapeHtml(item.venue)}</p>
      <p><strong>Speaker:</strong> ${escapeHtml(item.speaker)}</p>
      <p><strong>Attendees:</strong> ${escapeHtml(item.attendees)}</p>
    </div>
```

```
  `;
}

function renderStudents(item) {
  return `
    <div class="record-card">
      <h3>${escapeHtml(item.name)}</h3>
      <div class="record-meta">
        <span class="chip">${escapeHtml(item.department)}</span>
        <span class="chip">Sem ${escapeHtml(item.semester)}</span>
      </div>
      <p><strong>CGPA:</strong> ${escapeHtml(item.cgpa)}</p>
      <p><strong>Status:</strong> ${escapeHtml(item.status)}</p>
      <p><strong>Email:</strong> ${escapeHtml(item.email)}</p>
    </div>
  `;
}

function renderFaqs(item) {
  return `
    <div class="record-card">
      <h3>${escapeHtml(item.question)}</h3>
      <div class="record-meta">
        <span class="chip">${escapeHtml(item.category)}</span>
      </div>
      <p>${escapeHtml(item.answer)}</p>
    </div>
  `;
}

const renderers = {
  events: renderEvents,
  students: renderStudents,
  faqs: renderFaqs
};

export function renderRecords(container, items, datasetName) {
  if (!items.length) {
    container.innerHTML = '<div class="empty-state">No matching records found.</div>';
    return;
  }

  const renderer = renderers[datasetName] || ((item) => `<div class="record-
card"><h3>${escapeHtml(item.name || item.title || item.question)}</h3></div>`);
  container.innerHTML = items.map((item) => renderer(item)).join('');
}

export function renderPagination(container, currentPage, totalPages, onPageChange) {
```

```
  if (totalPages <= 1) {
    container.innerHTML = '';
    return;
  }

  const buttons = [];
  buttons.push(`<button type="button" ${currentPage === 1 ? 'disabled' : ''} data-
page="${currentPage - 1}">Prev</button>`);

  for (let page = 1; page <= totalPages; page += 1) {
    buttons.push(
      `<button type="button" class="${page === currentPage ? 'active' : ''}" data-
page="${page}">${page}</button>`
    );
  }

  buttons.push(`<button type="button" ${currentPage === totalPages ? 'disabled' : ''}
data-page="${currentPage + 1}">Next</button>`);

  container.innerHTML = buttons.join('');
  container.querySelectorAll('button[data-page]').forEach((button) => {
    button.addEventListener('click', () => onPageChange(Number(button.dataset.page)));
  });
}
```

4.4 Search and Filter Logic

```
function applySearchAndFilter() {
  const query = state.searchTerm.trim().toLowerCase();
  let nextItems = [...state.data];

  if (query) {
    nextItems = nextItems.filter((item) =>
JSON.stringify(item).toLowerCase().includes(query));
  }

  if (state.filterValue !== 'all') {
    const field = getFilterFieldForDataset(state.dataset);
    nextItems = nextItems.filter((item) => String(item[field]) ===
String(state.filterValue));
  }

  if (state.sortValue !== 'default') {
    nextItems = [...nextItems].sort((a, b) => {
      const aValue = a[state.sortValue];
      const bValue = b[state.sortValue];
      if (typeof aValue === 'number' && typeof bValue === 'number') {
        return bValue - aValue;
      }
```

```
      return String(aValue ?? '').localeCompare(String(bValue ?? ''));
    });
  }

  state.filteredData = nextItems;
  state.currentPage = 1;
}
```

4.5 Sorting and Pagination Logic

```
import { fetchDataset, getFilterValues, getSortOptions } from './dataService.js';
import { renderPagination, renderRecords } from './ui.js';

const state = {
  dataset: 'events',
  data: [],
  filteredData: [],
  searchTerm: '',
  filterValue: 'all',
  sortValue: 'default',
  currentPage: 1,
  pageSize: 5,
  loading: false,
  error: ''
};

const datasetTabs = document.querySelector('#datasetTabs');
const searchInput = document.querySelector('#searchInput');
const filterSelect = document.querySelector('#filterSelect');
const sortSelect = document.querySelector('#sortSelect');
const pageSizeSelect = document.querySelector('#pageSizeSelect');
const statusBox = document.querySelector('#statusBox');
const recordsGrid = document.querySelector('#recordsGrid');
const pagination = document.querySelector('#pagination');

function showStatus(message, type = 'success') {
  statusBox.textContent = message;
  statusBox.className = `status-box ${type}`;
}

function getVisibleRecords() {
  const startIndex = (state.currentPage - 1) * state.pageSize;
  return state.filteredData.slice(startIndex, startIndex + state.pageSize);
}

function applySearchAndFilter() {
  const query = state.searchTerm.trim().toLowerCase();
  let nextItems = [...state.data];
```

```
  if (query) {
    nextItems = nextItems.filter((item) =>
JSON.stringify(item).toLowerCase().includes(query));
  }

  if (state.filterValue !== 'all') {
    const field = getFilterFieldForDataset(state.dataset);
    nextItems = nextItems.filter((item) => String(item[field]) ===
String(state.filterValue));
  }

  if (state.sortValue !== 'default') {
    nextItems = [...nextItems].sort((a, b) => {
      const aValue = a[state.sortValue];
      const bValue = b[state.sortValue];
      if (typeof aValue === 'number' && typeof bValue === 'number') {
        return bValue - aValue;
      }
      return String(aValue ?? '').localeCompare(String(bValue ?? ''));
    });
  }

  state.filteredData = nextItems;
  state.currentPage = 1;
}

function getFilterFieldForDataset(datasetName) {
  if (datasetName === 'events') return 'category';
  if (datasetName === 'students') return 'department';
  return 'category';
}

function updateFilterOptions() {
  const fieldName = getFilterFieldForDataset(state.dataset);
  const options = getFilterValues(state.data, fieldName);

  const previousValue = state.filterValue;
  filterSelect.innerHTML = '<option value="all">All</option>' + options.map((value) =>
`<option value="${value}">${value}</option>`).join('');

  if (options.includes(previousValue)) {
    state.filterValue = previousValue;
  } else {
    state.filterValue = 'all';
  }

  filterSelect.value = state.filterValue;
```

```
}

function updateSortOptions() {
  const options = getSortOptions(state.dataset);
  sortSelect.innerHTML = options.map((option) => `<option
value="${option.value}">${option.label}</option>`).join('');
  sortSelect.value = state.sortValue;
}

function renderDatasetButtons() {
  const datasets = [
    { key: 'events', label: 'Events' },
    { key: 'students', label: 'Students' },
    { key: 'faqs', label: 'FAQs' }
  ];

  datasetTabs.innerHTML = datasets
    .map(
      (dataset) =>
        `<button class="dataset-tab ${state.dataset === dataset.key ? 'active' : ''}"
type="button" data-dataset="${dataset.key}">${dataset.label}</button>`
    )
    .join('');

  datasetTabs.querySelectorAll('.dataset-tab').forEach((button) => {
    button.addEventListener('click', () => switchDataset(button.dataset.dataset));
  });
}

function render() {
  const totalPages = Math.max(1, Math.ceil(state.filteredData.length /
state.pageSize));
  const visibleRecords = getVisibleRecords();

  renderRecords(recordsGrid, visibleRecords, state.dataset);
  renderPagination(pagination, state.currentPage, totalPages, (page) => {
    state.currentPage = Math.min(Math.max(1, page), totalPages);
    render();
  });
}

function switchDataset(datasetName) {
  state.dataset = datasetName;
  state.sortValue = 'default';
  state.filterValue = 'all';
  state.searchTerm = '';
  searchInput.value = '';
  loadDataset(datasetName);
```

```
}

async function loadDataset(datasetName) {
  try {
    state.loading = true;
    state.error = '';
    showStatus('Loading data...', 'success');
    const items = await fetchDataset(datasetName);
    state.data = items;
    state.dataset = datasetName;
    updateFilterOptions();
    updateSortOptions();
    applySearchAndFilter();
    renderDatasetButtons();
    render();
    showStatus(`Loaded ${items.length} records from ${datasetName}.`, 'success');
  } catch (error) {
    state.error = error.message;
    showStatus('Data not loaded. Please check the file or try again.', 'error');
    recordsGrid.innerHTML = '<div class="empty-state">Data could not be
loaded.</div>';
    pagination.innerHTML = '';
  } finally {
    state.loading = false;
  }
}

searchInput.addEventListener('input', (event) => {
  state.searchTerm = event.target.value;
  applySearchAndFilter();
  render();
});

filterSelect.addEventListener('change', (event) => {
  state.filterValue = event.target.value;
  applySearchAndFilter();
  render();
});

sortSelect.addEventListener('change', (event) => {
  state.sortValue = event.target.value;
  applySearchAndFilter();
  render();
});

pageSizeSelect.addEventListener('change', (event) => {
  state.pageSize = Number(event.target.value) || 5;
  state.currentPage = 1;
```

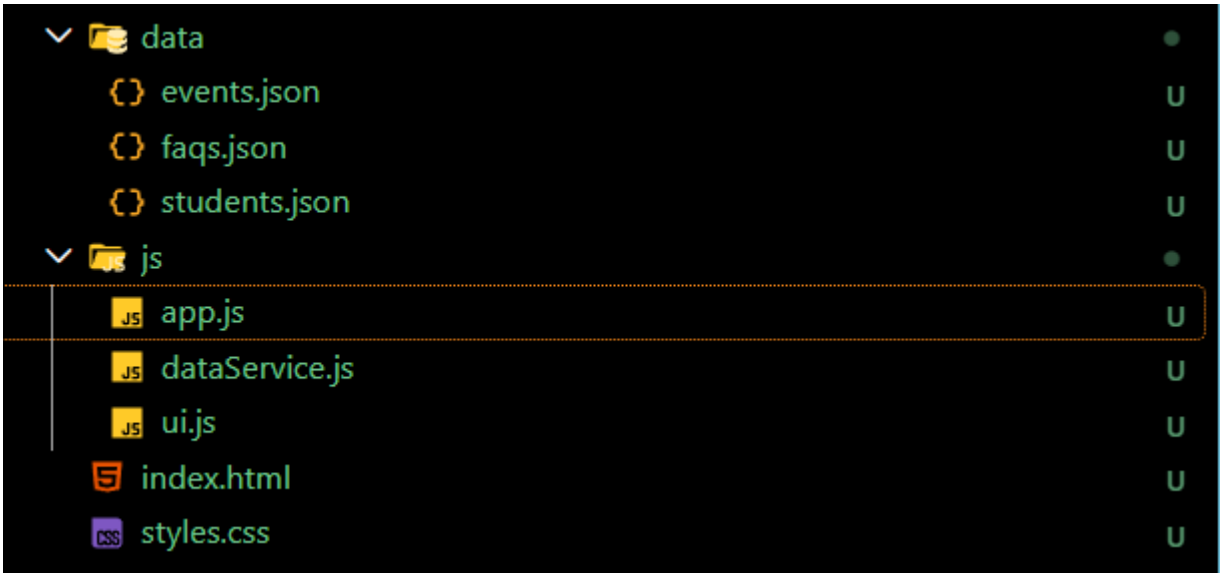
```
  render();
});

renderDatasetButtons();
updateSortOptions();
loadDataset(state.dataset);
```

4.6 Loading and Error Handling

```
async function loadDataset(datasetName) {
  try {
    state.loading = true;
    state.error = '';
    showStatus('Loading data...', 'success');
    const items = await fetchDataset(datasetName);
    state.data = items;
    state.dataset = datasetName;
    updateFilterOptions();
    updateSortOptions();
    applySearchAndFilter();
    renderDatasetButtons();
    render();
    showStatus(`Loaded ${items.length} records from ${datasetName}.`, 'success');
  } catch (error) {
    state.error = error.message;
    showStatus('Data not loaded. Please check the file or try again.', 'error');
    recordsGrid.innerHTML = '<div class="empty-state">Data could not be loaded.</div>';
    pagination.innerHTML = '';
  } finally {
    state.loading = false;
  }
}
```

4.7 Modular JavaScript Structure



5. Result

The webpage fetches and displays JSON data dynamically and provides search, filter, sorting, and pagination features. Loading and error handling are included, and the JavaScript is organized into modules. The JSON datasets contain at least 15 records each, as required.

6. Conclusion

This practical demonstrates how JSON files and the Fetch API can load data into a webpage, how JavaScript can transform and present it interactively, and how modular code structure and loading/error states improve maintainability and usability.

— End of Practical 6 Submission —